ASSIGNMENT 18-4

Task 1 – Movie Database API Integration
Scenario
You are building a movie recommendation tool that fetches movie details from a public Movie Database API.

```python
import requests
import json

def get_movie_details(title):
    API_KEY = "YOUR_API_KEY"
    API_ENDPOINT = "http://www.omdbapi.com/"

    if API_KEY == "YOUR_API_KEY" or API_ENDPOINT == "http://www.omdbapi.com/":
        print("Error: Please replace YOUR_API_KEY and API_ENDPOINT with actual values.")
        return

    params = {
        "t": title,  # Movie title
        "apikey": API_KEY
    }
    try:
        response = requests.get(API_ENDPOINT, params=params, timeout=5)
        response.raise_for_status()  # Raise an HTTPError for bad responses (4xx or 5xx)

        data = response.json()

        if data.get("Response") == "True":
            print(f"\n--- Movie Details: {data.get('Title')} ---")
            print(f"Title: {data.get('Title', 'N/A')}")
            print(f"Release Year: {data.get('Year', 'N/A')}")
            print(f"Rating: {data.get('imdbRating', 'N/A')} (IMDb)")
            print(f"Plot Summary: {data.get('Plot', 'N/A')}")
        elif data.get("Response") == "False":
```

EXPLANATION:
IP validation ensures correct IPv4 format before API call.
API integration fetches country, city, and ISP details.
Error handling covers:

- Invalid IP format
- API/network failures
- JSON parsing errors

```
        except requests.exceptions.Timeout:
            print("Error: The request timed out. Please check your internet connection or try again later
        except requests.exceptions.ConnectionError:
            print("Error: Could not connect to the API. Please check your internet connection.")
        except requests.exceptions.HTTPError as e:
            print(f"Error: HTTP Request failed: {e}")
            if response.status_code == 401:
                print("Possible API key error. Please check your API key.")
        except json.JSONDecodeError:
            print("Error: Could not decode JSON response. The API might be returning invalid data.")
        except Exception as e:
            print(f"An unexpected error occurred: {e}")
    movie_title = input("Enter movie title: ")
    get_movie_details(movie_title)

•••  Enter movie title: H
     Error: Please replace YOUR_API_KEY and API_ENDPOINT with actual values.
```

Task 4 – Geolocation API (IP Address Lookup)

Scenario

A cybersecurity tool needs to detect the geographical location of a user based on IP address.

EXPLANATION:

IP validation ensures correct IPv4 format before API call.
API integration fetches country, city, and ISP details.
Error handling covers:

- Invalid IP format
- API/network failures
- JSON parsing errors

```python
import requests
import re

# ------------------------------
# 1. Validate IP Address
# ------------------------------
def is_valid_ip(ip):
    pattern = r"^(?:[0-9]{1,3}\.){3}[0-9]{1,3}$"
    if re.match(pattern, ip):
        return all(0 <= int(part) <= 255 for part in ip.split("."))
    return False

# ------------------------------
# 2. Fetch Geolocation Data
# ------------------------------
def get_geolocation(ip):
    if not is_valid_ip(ip):
        return {"error": "Invalid IP format"}

    try:
        # Example API (free)
        url = f"http://ip-api.com/json/{ip}"
        response = requests.get(url, timeout=5)

        # Check API response status
        if response.status_code != 200:
```

```python
            return {
                "ip": ip,
                "country": data.get("country"),
                "city": data.get("city"),
                "isp": data.get("isp")
            }

        except requests.exceptions.RequestException:
            return {"error": "Network/API connection error"}

# ------------------------------
# 3. Example Usage
# ------------------------------
if __name__ == "__main__":
    ip_input = input("Enter IP address: ")
    result = get_geolocation(ip_input)
    print(result)
```

```
Enter IP address:
```

Task 5 – Real-Time Application: Payment Gateway Simulation API

Scenario

You are building a test payment integration system that connects to a sandbox payment API.

```python
import requests
import logging
from datetime import datetime

# ------------------------------
# 1. Setup Logging
# ------------------------------
logging.basicConfig(
    filename="failed_transactions.log",
    level=logging.ERROR,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

# ------------------------------
# 2. Input Validation
# ------------------------------
def validate_payment(amount, currency, payment_method):
    valid_methods = ["card", "upi", "netbanking", "wallet"]

    if amount <= 0:
        return False, "Amount must be greater than 0"

    if not currency.isalpha() or len(currency) != 3:
        return False, "Invalid currency format (use 3-letter code like USD, INR)"

    if payment_method.lower() not in valid_methods:
        return False, f"Invalid payment method. Choose from {valid_methods}"
```

[5]
30s

```python
# ------------------------------
# 3. Payment Processing Function
# ------------------------------
def process_payment(amount, currency, payment_method):
    is_valid, message = validate_payment(amount, currency, payment_method)

    if not is_valid:
        return {"status": "failed", "reason": message}

    url = "https://jsonplaceholder.typicode.com/posts"  # Mock API

    payload = {
        "amount": amount,
        "currency": currency.upper(),
        "payment_method": payment_method.lower()
    }

    try:
        response = requests.post(url, json=payload, timeout=5)

        # ------------------------------
        # Handle HTTP Errors
        # ------------------------------
        if response.status_code == 200 or response.status_code == 201:
            return {
                "status": "success",
                "message": "Payment processed successfully",
```

```python
        elif response.status_code == 400:
            error_msg = "Bad Request (Invalid payment details)"
        elif response.status_code == 401:
            error_msg = "Unauthorized (Invalid API credentials)"
        elif response.status_code == 500:
            error_msg = "Server Error (Try again later)"
        else:
            error_msg = f"Unexpected HTTP error: {response.status_code}"

        # Log failure
        logging.error(f"Payment Failed | Payload: {payload} | Error: {error_msg}")

        return {"status": "failed", "reason": error_msg}

    except requests.exceptions.Timeout:
        logging.error(f"Timeout Error | Payload: {payload}")
        return {"status": "failed", "reason": "Request timed out"}

    except requests.exceptions.ConnectionError:
        logging.error(f"Connection Error | Payload: {payload}")
        return {"status": "failed", "reason": "Network connection error"}

    except requests.exceptions.RequestException as e:
        logging.error(f"General Error | Payload: {payload} | Error: {str(e)}")
        return {"status": "failed", "reason": "Unexpected error occurred"}

# ------------------------------
# 4. Run Example
```

```python
# ------------------------------
# 4. Run Example
# ------------------------------
if __name__ == "__main__":
    try:
        amount = float(input("Enter amount: "))
        currency = input("Enter currency (e.g., INR, USD): ")
        payment_method = input("Enter payment method (card/upi/netbanking/wallet): ")

        result = process_payment(amount, currency, payment_method)

        print("\nTransaction Result:")
        print(result)

    except ValueError:
        print("Invalid input! Amount must be a number.")
```

```
Enter amount: 500
Enter currency (e.g., INR, USD): 2
Enter payment method (card/upi/netbanking/wallet): SBI

Transaction Result:
{'status': 'failed', 'reason': 'Invalid currency format (use 3-letter code like USD, INR)'}
```

EXPLANATION:

The script **validates inputs** (amount, currency, payment method) before processing.
It sends a **POST request** to a mock payment API to simulate a transaction.
It handles errors like **invalid input, HTTP errors (400, 401, 500), and network issues**.
Successful payments show a confirmation message.
Failed transactions are saved in a log file for tracking and debugging.